JUDGEAI ? USER GUIDE (API v2.0)

OVERVIEW
JudgeAI ingests judgment PDFs, stores them in Supabase Storage, extracts structured fields via Groq (LLaMA 3.x), derives a government-oriented action_plan (priorities, compliance vs appeal posture, deadlines), optionally classifies department with embeddings, embeds cases for semantic search, and supports officer/admin verification workflows.

SECTION A ? WHAT YOU NEED

? Python 3 with project virtualenv: .venv (install: pip install -r requirements.txt)
? Node.js 18+ for the Vite/React frontend under folder: frontend/
? File .env in project root containing at minimum:
  SUPABASE_URL, SUPABASE_KEY, GROQ_API_KEY,
  optionally SUPABASE_STORAGE_BUCKET (defaults to court-judgments)
? In Supabase SQL editor, execute: backend/db_migrations_upgrade_202602.sql
  This adds pgvector, cases.layout_blocks and cases.embedding(384),
  extracted_actions.action_plan + action_plan_reasoning, permissive notifications.user_id,
  and function match_cases_semantic(...) for cosine search.

SECTION B ? HOW TO START SERVERS (Windows / PowerShell)

Terminal 1 ? Backend (repository root):
  Set-Location d:\JudgeAI
  .venv\Scripts\uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000

Terminal 2 ? Frontend:
  Set-Location d:\JudgeAI\frontend
  npm run dev

URLs: API http://127.0.0.1:8000 | Swagger UI http://127.0.0.1:8000/docs
     UI http://localhost:5173 (Vite default)

SECTION C ? FRONTEND SCREENS & USER FLOW
---
1) Home
   ? Drag one PDF or click to upload.
   ? Single file: uploads to storage, then calls extract ? shows extraction cards + fused
     confidence preview and action_plan JSON preview when returned.
   ? Multiple PDFs: calls batch upload ? shows job id; server processes in background.

2) Officer dashboard
   ? Counts: pending / approved / completed extractions, urgent deadlines, recent uploads.

3) Verification queue
   ? List and open cases; approve, edit fields, or reject with reason; audit trail.

4) Case details
   ? Metadata, fused radial confidence gauge (LLM + timeline + department + appeal).
   ? PDF viewer: highlights from layout (directive / deadline / party cues) when blocks
     exist; otherwise plain iframe to public PDF URL.
   ? Government action plan summary + explainability text (why appeal, department, deadline).

5) Admin dashboard
  ? Analytics charts, deadline alerts, activity log, CSV export, officer creation.
  ? Government intelligence widgets: appeal-recommended cases, compliance-required,
    deadlines within 7 days, department pending backlog.

6) Realtime toasts (optional)
  ? Subscribes to new rows in notifications for deadline_imminent when RLS allows.

SECTION D ? API ENDPOINTS, INPUTS, AND OUTPUTS

GET /
    Output: { status, service, version }

POST /api/upload-pdf
    Input: multipart file (PDF), form uploaded_by (optional).
    Output: message, case_number, pdf_url, metadata, db_record (cases insert).

POST /api/upload-batch
    Input: multipart files[] (multiple PDFs), form uploaded_by.
    Output: job_id, files_enqueued, message.
    Side effect: uploads to storage, inserts cases, runs extraction per file in background.

GET /api/batch-status/{job_id}
    Output: status, timestamps, files[], successes[], errors[] when completed.

POST /api/extract-actions
    Input: JSON { "pdf_url": "<public supabase url>" }.
    Output: extracted_data (LLM), action_plan, action_plan_reasoning, status, db_record.
    Side effect: insert extracted_actions; update cases.layout_blocks and embedding when
    columns exist.

POST /api/demo-process
    Input: JSON { "pdf_url": "..." }.
    Output: final_action_plan, reasoning, extracted_data, auto_approved_action_id.
    Side effect: same persistence as extract + sets status approved on that row.

POST /api/approve-action/{id}   Input: optional approved_by ? status approved + audit.
POST /api/edit-action/{id}   Input: field updates + edited_by ? status edited + audit.
POST /api/reject-action/{id}   Input: rejection_reason + rejected_by ? rejected + audit.

GET /api/officer-dashboard   Query: department (optional).
    Output: pending_cases, approved_cases, completed_cases, urgent_deadlines,
          total_extracted, recent_uploads.

GET /api/admin-dashboard
    Output: total_cases, status_distribution, deadline_alerts, department_stats,
          verification_counts, recent_activities, verification_accuracy_trend,
          cases_processed_per_department, appeal_recommended_cases,
          compliance_required_cases, upcoming_deadlines_7_days,
          department_pending_breakdown.

GET /api/cases
    Query: skip, limit, status, department, case_number,
         priority_level, action_type, deadline_range_min_days, deadline_range_max_days.
    Output: { total, data[], skip, limit } ? filters use action_plan when present.

GET /api/cases/{action_id}
    Output: action (full row), deadline_info, audit_logs, layout_blocks[], pdf_highlights[],

GET /api/search-semantic?q=&limit=
    Output: { query, limit, results[] } with id, case_number, pdf_url, similarity;
    if RPC missing, note + text fallback on case_number.

SECTION E ? BACKGROUND ENGINES (WHAT EACH PRODUCES)

? LLM extraction: structured JSON for standard judgment fields + source_sentence.
? Timeline parser: relative phrases (weeks/days/immediately/reasonable time) ? offset +
  inferred deadline date + confidence.
? Appeal recommender: YES / NO / REVIEW + limitation days (civil 90 vs service-ish 30).
? Department classifier: MiniLM embedding vs department profiles; may override LLM dept
  when confidence > 0.75.
? Confidence fusion: single 0?1 score from weighted LLM + timeline + department + appeal.
? Layout blocks: PyMuPDF text blocks with bbox per page for UI highlights.
? Embedding service: 384-d vector on case text for semantic search.

SECTION F ? RECOMMENDED SMOKE TEST FILE
---
backend/fixtures/sample_judgment_rte_fixture.pdf ? synthetic text PDF with education/RTE
context, compliance language, timelines, and appeal language for pipeline exercise.

Document generated automatically for JudgeAI repository.